The Journal of Open Source Software

formHTR: Template-Guided OCR Extraction of Handwritten Content from Scanned Forms

Matej Troják [1¶], **Stéphane Pesant** [2], **Jan Glos**[3], **Kerstin Leberecht** [1], **and Michael Kuhn** [1]

1 EMBL, Heidelberg, Germany 2 EMBL-EBI, Hinxton, UK 3 Masaryk university, Czech republic ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Neea Rusch

Reviewers:

- @robertoffmoura
- @dhruvsekhawat

Submitted: 11 June 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

`formHTR` is a Python software package for automatic handwritten text recognition (HTR) and extraction from scanned form documents. While the content extraction itself is accomplished using advanced pretrained optical character recognition (OCR) models, a key step in the process is considering the prior knowledge of the expected contents. That is achieved by the precise, semi-automatic annotation of the regions of interest (ROIs) in the form template, specifying the locations and content types of ROIs. Using this approach in combination with comparing and evaluating outputs from multiple OCR models is designed to support more reliable extraction.

Statement of need

Large-scale scientific expeditions often collect huge amounts of samples, with a need to record the origin and context of the samples and observed features (metadata). While digital metadata collection methods are becoming more popular, paper forms (so-called *logsheets*) are still the most popular method for their reliability in extreme environments, stability, natural scalability, and ease of use (Brewer & Guiterman, 2016; van Tamelen, 2004).

It is necessary to build an infrastructure that processes logsheets automatically with a minimal amount of manual interventions and laborious proofreading. OCR methods (Mori et al., 1992) are used to extract the content from a scanned logsheet, with additional complexity added by the handwritten nature of the content. While training custom models on a particular handwriting style of a person is generally more precise, this approach is impractical for large-scale expeditions with high turnaround of staff and consequently higher amount of distinct handwritings. The use of multiple pretrained general purpose OCR models, allowing consensus or majority decision making, is a more suitable approach.

Just as large-scale expeditions enforce standards on sample collection, they also enforce them on the metadata level. That means standardised logsheets are often developed, and used repeatedly in various sampling scenarios. As a consequence, to digitalise the contents of such logsheets, we can leverage their known structure and expected content types to navigate the OCR methods for more reliable and precise results.

State of the field

The current OCR landscape of pretrained tools can be split between open-source models and cloud-based services. On the open-source side, there are many tools such as Tesseract OCR (Google & HP, 2021), EasyOCR (Jaided AI, 2024), OCR4All (Reul et al., 2019), and

DRAFT

PaddleOCR (Cui et al., 2025) which provide pretrained models or optionally allow to train custom models. Some of the trade-offs are in accuracy, speed, and deployment complexity. Cloud-based pretrained OCR services, including Google Cloud Vision API (Google LLC, 2024), Azure AI Vision (Microsoft Corporation, 2024), and Amazon Textract (Amazon Web Services, 2024), are accessible via APIs and are highly optimised for printed and handwritten text, with some also providing higher-level outputs (e.g. key–value pairs).

Naturally, the combination of multiple tools, models, or services has emerged. Systems such as OCRmyPDF (Barlow, 2026) or unified interfaces like OcrPy (Maxentlabs, 2022) integrate engines like Tesseract, cloud APIs, and downstream processing into a single pipeline, effectively abstracting over multiple OCR backends. The tool Handprint (Hucka, 2022) combines multiple cloud services and outputs annotated images or raw results, as well as compares the recognised text against the expected content[1]. None of these tools integrates a reusable form template, aligns a scan to that template, assigns recognised fragments to named fields, or outputs a table with fused results from several engines by the voting process. Extending any of them would have meant replacing that output model rather than reusing it, so a dedicated extraction pipeline was a more straightforward design.

Software design

`formHTR` is structured into two main steps with an overview in Figure 1. The specification step selects regions of interest (ROIs) from a template and assigns them a meaning. This is achieved by selecting and manipulating the region locations, while the variable names and ROI types are assigned to individual ROIs. The output of the specification step is a config file containing position, name, and type of identified ROIs.

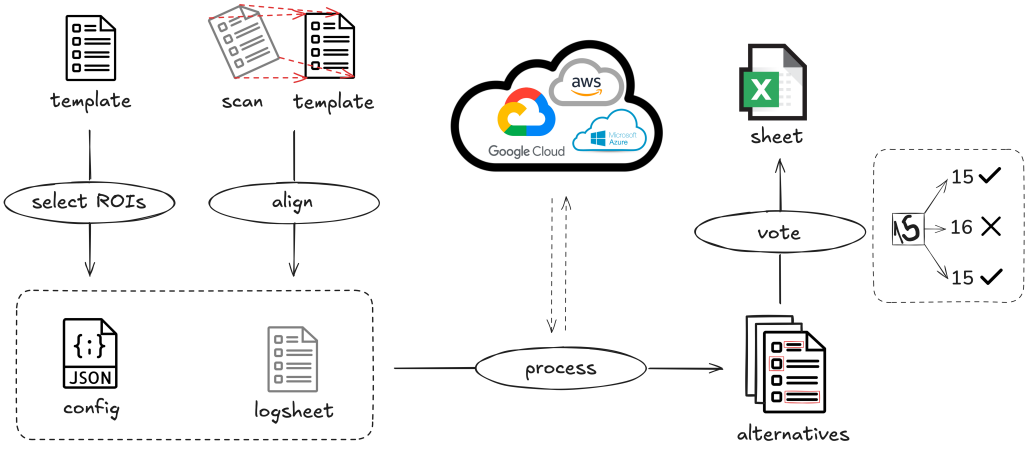


Figure 1: Schematic overview of formHTR annotation and processing workflow.

The next step identifies and extracts content from the ROIs. First, the scanned logsheet and its template are aligned to ensure the ROIs actually match the regions in the scanned logsheet (otherwise they would point to potentially empty or generally mismatched regions). While the goal is straightforward, the execution can be problematic, especially when the template has no fiducial markers or the scan quality is low. For this reason, a manual alignment is possible as well.

The aligned logsheet is converted to an image and several OCR models are queried to identify and extract the content. For this purpose, three services are used by calling their respective APIs - Google Cloud Vision (Google LLC, 2024), Azure AI Vision Read API (Microsoft Corporation,

[1]The package is not maintained anymore.

DRAFT

2024), and Amazon Textract (Amazon Web Services, 2024). Cloud APIs were preferred to open-source engines because they provide pretrained handwriting recognition without the need to train or host models at expedition scale. Users accept an external dependency, setup and possible usage costs[2], and the privacy implications of sending scans to third parties; they can enable any subset of the three services, or later add self-hosted backends. All three services output a set of detected words with their location (bounding box) in the image.

After collecting the identified contents for each service, the process of binning assigns each captured text fragment (word) to the appropriate ROI. Several cases need to be considered, such as a word spanning over multiple ROIs, multiple words overlapping with a single ROI, or a sentence split into several words (see examples in Figure 2). To find all overlaps for a ROI from all the services, we use Rtree (Gillies, 2025) to index the regions and capture their overlaps for all services' outputs and the template specification, and consequently use it to effectively query for intersections. Additionally, the logsheet can contain preprinted contents (*residuals*) that do not belong to ROIs, but can be shifted to its boundary box by an imperfect alignment. This is handled by defining the residuals already in the specification step, thus allowing to exclude them from the outputs.

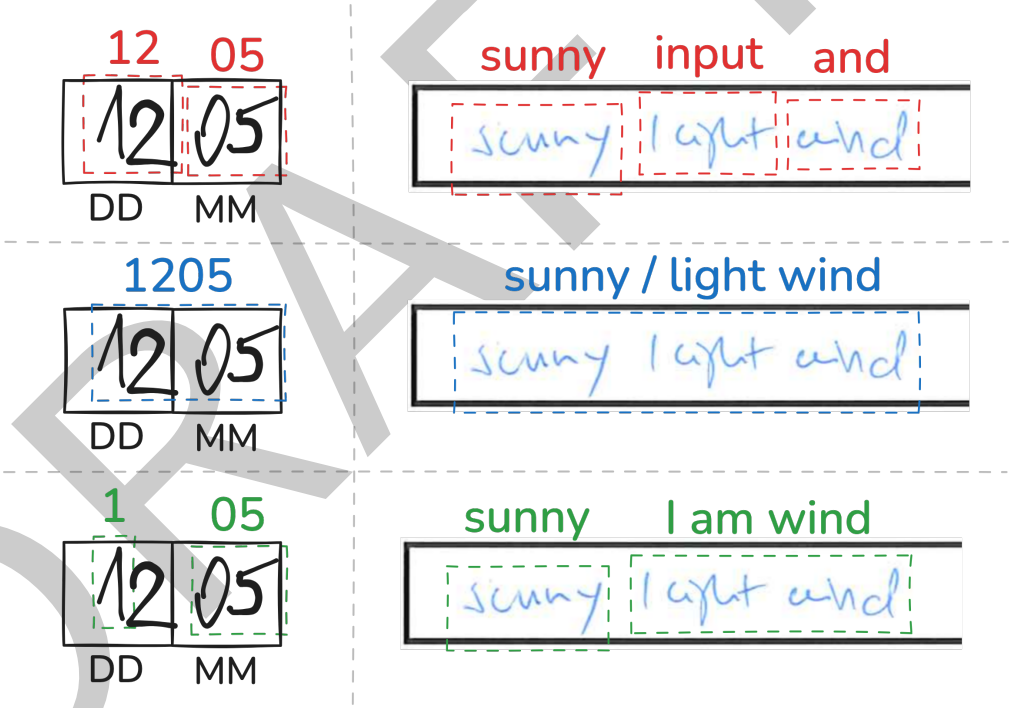


Figure 2: Examples of different scenarios of text position identification relative to a ROI. Each row (distinguished by colour) corresponds to a different service. In the left column, two regions are in close proximity, causing the identified text to overlap with a neighbouring region or to be recognised as a single word. In the right column, the detected words and sentence splitting into words vary across services.

Assuming the words are binned for all services, a majority vote is applied character by character after pairwise alignment of the readings. Ties are broken deterministically by keeping the first service in a fixed order. If a field spans more than three lines, or more than five words on a line, voting is skipped and the first service that returned content is used. Numeric ROIs prefer a numeric parsing of the readings when one is available. The tool works with fewer than

[2]All services offer a free tier with limited amount of requests per month. The user needs to provide suitable account credentials. A quick start description how to do this is available at the wiki pages of the `formHTR` repository.

DRAFT

three services enabled, with correspondingly less opportunity to resolve disagreements. A more elaborate consensus would need comparable confidence scores or labelled data, which we do not assume, thus implementing a richer rule remains a natural future extension (e.g. allow only values from a dictionary, only content satisfying a regex, only numbers from an integer range, or content satisfying a length threshold).

The output of `formHTR` is an Excel spreadsheet (an `.xlsx` file) with two sheets. `Metadata` sheet has three columns - a variable name coming from the specification, the extracted content as the result of the voting algorithm, and a picture cut out of the scanned (and aligned) logsheet based on the bounding box defined in the specification. This can be used for a quick proofreading of the outputs. `Extra` sheet contains any miscellaneous content that neither falls into any ROI nor was filtered out as a residual. This can typically alert the user to any comments and notes written on unexpected parts of the logsheet.

Research impact statement

The need for the `formHTR` tool arose during the TRaversing European Coastlines (TREC) expedition[3]. The tool was developed and continuously optimised for the expedition, and consequently used on a collection of approximately ten thousand double-sided logsheets spanning around a hundred different template types, together providing contextual metadata to more than eighty thousand collected samples. Using `formHTR`, the logsheet templates were annotated and all the scanned logsheets processed. The metadata was extracted and curated, and archived[4] in the BioSamples database (Courtot et al., 2022). The contextual metadata is an essential foundation for sample analysis and data production, which, at the time of writing, is in the initial phase.

Example workflow

The following is an example workflow in which we first create and annotate ROIs for the template, and consequently process the scanned logsheet, ultimately creating the output Excel spreadsheet.

```
# 1) Create ROI config for a template
formhtr select-rois --pdf-file template.pdf --output-file config.json

# 2) Annotate ROI types and variable names
formhtr annotate-rois --pdf-file template.pdf --config-file config.json --
output-file config_annotated.json

# 3) Process a scanned logsheet
formhtr process-logsheet \
  --pdf-logsheet scan.pdf \
  --pdf-template template.pdf \
  --config-file config_annotated.json \
  --output-file output.xlsx \
  --google google_credentials.json \
  --amazon amazon_credentials.json \
  --azure azure_credentials.json
```

[3] https://www.embl.org/about/info/trec/

[4] https://www.ebi.ac.uk/biosamples/samples?text=Traversing+European+Coastlines+%28TREC%29+expedition

DRAFT

Author Contributions

MT wrote the manuscript and developed the software. JG contributed to the software. SP, MK, and KL contributed via conceptual guidance and contributed to the manuscript.

AI usage disclosure

Generative AI assistance was used through the Cursor IDE with automatic agent selection, so individual model names and versions were not recorded. It was used to draft tests and documentation, and during JOSS review to help revise this manuscript and author responses. It was not used to design the extraction pipeline or to write the original manuscript. Human authors reviewed, edited, and validated all AI-assisted outputs and made the core design decisions.

Acknowledgements

We thank the late Peer Bork for general supervision of this work. This publication was enabled by the support of EMBL member states to the TREC expedition (within the framework of EMBL's Molecules to Ecosystems Programme 2022-2026) and partially funded by the European Union's Horizon 2020 research and innovation programme (project BIOcean5D with grant agreement No. 101059915). Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

References

Amazon Web Services. (2024). *Amazon textract*. https://aws.amazon.com/textract.

Barlow, J. R. (2026). *OCRmyPDF* (Version 17.4.1). https://pypi.org/project/ocrmypdf/

Brewer, P. W., & Guiterman, C. H. (2016). A new digital field data collection system for dendrochronology. *Dendrochronologia*, *38*, 131–135. https://doi.org/10.1016/j.dendro.2016.04.005

Courtot, M., Gupta, D., Liyanage, I., Xu, F., & Burdett, T. (2022). BioSamples database: FAIRer samples metadata to accelerate research data management. *Nucleic Acids Research*, *50*(D1), D1500–D1507. https://doi.org/10.1093/nar/gkab1046

Cui, C., Sun, T., Lin, M., Gao, T., Zhang, Y., Liu, J., Wang, X., Zhang, Z., Zhou, C., Liu, H., Zhang, Y., Lv, W., Huang, K., Zhang, Y., Zhang, J., Zhang, J., Liu, Y., Yu, D., & Ma, Y. (2025). *PaddleOCR 3.0 technical report*. Baidu Inc. https://arxiv.org/abs/2507.05595

Gillies, S. (2025). *Rtree: Spatial indexing for Python* (Version 1.4.1). https://pypi.org/project/rtree/

Google LLC. (2024). *Google cloud vision API*. https://cloud.google.com/vision.

Google, & HP. (2021). *Tesseract OCR engine* (Version 5.5.4). /url%7Bhttps://github.com/tesseract-ocr/tesseract%7D

Hucka, M. (2022). *Handprint: A tool for comparing handwriting recognition services* (Version 1.6.0). https://doi.org/10.22002/D1.20207

Jaided AI. (2024). *EasyOCR: End-to-end multi-lingual optical character recognition (OCR) solution* (Version 1.7.2). https://pypi.org/project/easyocr/

Maxentlabs. (2022). *OcrPy: OCR, archive, index and search any documents with ease* (Version 0.3.10). https://pypi.org/project/ocrpy/

DRAFT

Microsoft Corporation. (2024). *Azure Cognitive Services Computer Vision Client Library for Python* (Version 0.9.1). https://pypi.org/project/azure-cognitiveservices-vision-computervision/

Mori, S., Suen, C. Y., & Yamamoto, K. (1992). Historical review of OCR research and development. *Proceedings of the IEEE*, *80*(7), 1029–1058. https://doi.org/10.1109/5.156468

Reul, C., Christ, D., Hartelt, A., Balbach, N., Wehner, M., Springmann, U., Wick, C., Grundig, C., Büttner, A., & Puppe, F. (2019). OCR4all—an open-source tool providing a (semi-)automatic OCR workflow for historical printings. *Applied Sciences*, *9*(22). https://doi.org/10.3390/app9224853

van Tamelen, P. G. (2004). A comparison of obtaining field data using electronic and written methods. *Fisheries Research*, *69*(1), 123–130. https://doi.org/10.1016/j.fishres.2004.01.006

DRAFT